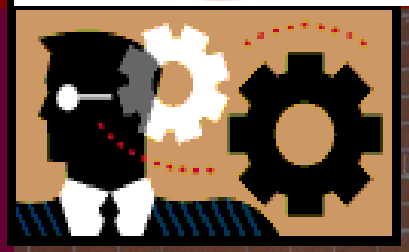


《图论与组合优化》

第二讲

排列组合生成算法

第二讲: 排列组合的生成算法

组合数学的主要问题:

(1) 存在: 满足一定条件配置的存在性.

(2) 计数: 计算出满足条件配置的数目.

(3) 算法: 构造所有配置的算法.

(4) 优化: 优化算法.

一. 排列生成算法

- 排列生成有几种典型算法, 这些算法都很有成效. 它们在实际中具有广泛应用价值.

1. 序数法
2. 字典序法
3. 邻位互换法(Johnson-Trotter)
4. 轮转法

1. 序数法

- 序数法基于一一对应概念.
- 先在排列和一种特殊的序列之间建立一种一一对应关系，然后再给出由序列产生排列的方法
- 因为序列的产生非常方便，这样我们就可以得到一种利用序列来生成排列的方法.
- 如何建立这种一一对应?

- 思路类似数的10进制、2进制和p进制表示.

$$n=\sum_{k=0}^{m-1}a_k 10^k,\ 0\le a_k\le 9;$$

$$n=\sum_{k=0}^{m-1}a_k 2^k,\ 0\le a_k\le 1;$$

$$n=\sum_{k=0}^{m-1}a_k p^k,\ 0\le a_k\le p-1.$$

- 这相当于自然数与某种序列之间建立了一一对应关系.
- 可以利用置换来表示整数:

$$n!=n(n-1)!=(n-1+1)(n-1)!$$

$$=(n-1)(n-1)!+(n-1)!$$

$$(n-1)!=(n-2)(n-2)!+(n-2)!$$

$$n!=(n-1)(n-1)!+(n-2)(n-2)!$$

$$+(n-3)(n-3)!+\ldots+2\cdot 2!+1\cdot 1!+1$$

- $n!-1=(n-1)\,(n-1)!+(n-2)\,(n-2)!$

  $+(n-3)\,(n-3)!+\ldots+2\cdot 2!+1\cdot 1!$

- 可以证明, 从0到$n!-1$之间的任何整数m都可唯一地表示为:

$$m=a_{n-1}\,(n-1)!+a_{n-2}\,(n-2)!+\ldots+a_2\cdot 2!+a_1\cdot 1!$$

其中$0\le a_i\le i$, $i=1,2,\ldots,n-1$.

- m与序列$(a_{n-1},a_{n-2},\ldots a_2,a_1)$一一对应
- 书中有确定这些系数的方法.
- 例如：$10=1\cdot 3!+2\cdot 2!+0\cdot 1!$

- 因为满足条件

$$0 \le a_i \le i,\ 1 \le i \le n-1 \qquad (2.1)$$

的序列

$$(a_{n-1}, a_{n-2}, \ldots, a_2, a_1)$$

共有$n!$个, 这恰好与0到$n!-1$的$n!$个整数一一对应.

- 需要建立满足条件(2.1)的$n!$个序列 $(a_{n-1}, a_{n-2}, \ldots, a_2, a_1)$和$n$元集合S的全部排列之间的一一对应关系.

- 还需要给出一种办法, 由每个满足条件(2.1)的序列$(a_{n-1},a_{n-2},\ \ldots,a_2,a_1)$可生成唯一的一个排列.
- 这样我们就可以产生出所有的排列.
- 怎么样由一个满足条件(2.1)的序列产生一个n阶排列?
- 如何把$1,2,\ldots,n$的一个排列与一个满足条件(2.1)的序列建立起直接的关系?

- 行列式定义中有逆序数的概念, 就是一个排列中违反自然顺序的数对: 比如12354的逆序数为1, 而43215的逆序数为6.
- 设$p_1p_2\cdots p_n$是任意一个n元排列, 则$i+1$后面比$i+1$小的数字的个数a_i总不超过i, 即$a_i \le i$, $i=1,2,\ldots,n-1$.
- 这样自然由一个排列得到一个序列 $(a_{n-1},a_{n-2},\ldots,a_2,a_1)$, 而且满足条件(2.1).

- 我们可以如下建立序列与排列的对应:
- 设序列 $(a_{n-1},a_{n-2},\ \ldots,a_2,a_1)$满足条件(2.1). 则它所对应的排列为$(p)=p_1p_2\cdots p_n$，其中$a_i$可以看作是排列$(p)$中数$i+1$所在位置后面比$i+1$小的数的个数.
- 要说明这种对应的合理性，必须清楚. 如何由序列产生出它所对应的排列.
- 我们通过一个具体的例题说明思想方法.

例2.1 (1) 4213 →(301)

4后面比4小的数的个数$a_3=3$；3后面比3小的数的个数$a_2=0$; 2后面比2小的数的个数$a_1=1$.

(2) (301) → 4213

由$a_3=3$知1,2,3都在4的后面; 由$a_2=0$知1,2都在3前面；由$a_1=1$知1在2后面.

(3) $(4213)\leftrightarrow(a_3a_2a_1)=(301)$.

- 比如其中的序列(221)所对应的排列:

  先由$a_3=2$决定4的位置

  再由$a_2=2$决定3的位置

  再由$a_1=1$决定2的位置

3	4	2	1

- 满足条件(2.1)的n!个序列很容易产生的, 利用这些序列就可以得到全体n阶排列.
- 这种利用序列产生排列的方法就是所谓的序数法.

- 利用序列得到相应排列是关键, 可以设想为给n个格子中填写1,2,…,n. 如上面的例题:

4	2	1	3

例2.2 设集合S={1,2,3,4}, 用序数法生成S的全部排列.

解 用序数法, 由各个序列对应生成的排列, 如表2.1所示.

N	$a_3a_2a_1$	$p_1p_2p_3p_4$	N	$a_3a_2a_1$	$p_1p_2p_3p_4$
0	000	1234	12	200	1423
1	001	2134	13	201	2413
2	010	1324	14	210	1432
3	011	2314	15	211	2431
4	020	3124	16	220	3412
5	021	3214	17	221	3421
6	100	1243	18	300	4123
7	101	2143	19	301	4213
8	110	1342	20	310	4132
9	111	2341	21	311	4231
10	120	3142	22	320	4312
11	121	3241	23	321	4321

2. 字典序法

对给定的字符集中的字符规定了一个先后关系，在此基础上规定两个全排列的先后是从左到右逐个比较对应的字符的先后。

[例]字符集{1, 2, 3}，较小的数字较先,这样按字典序生成的全排列是：

123, 132, 213, 231, 312, 321。

2. 字典序法

- 字典序法就是按照字典排序的思想逐一产生所有排列.
- 设想要得到由1,2,3,4以各种可能次序产生出4!个“单词”. 肯定先排1234, 再排1243, 下来是1324, 1342, …., 4321.
- 分析这种过程, 看如何由一个排列得到下一个排列, 并给出严格的数学描述.

例2.3 设有排列(p) =2763541, 按照字典式排序, 它的下一个排列是谁?

(q) =2764135.

(1) 2763541 [找最后一个正序35]

(2) 2763541 [找3后面比3大的最后一个数]

(3) 2764531 [交换3,4的位置]

(4) 2764135 [把4后面的531反序排列为135即得到最后的排列(q)]

- 求$(p)=p_1\cdots p_{i-1}p_i\cdots p_n$的下一个排列$(q)$:

(1) 求 $i=\max\{j \mid p_{j-1}<p_j\}$(找最后一个正序)

(2) 求 $j=\max\{k \mid p_{i-1}<p_k\}$(找最后大于$p_{i-1}$者)

(3) 互换p_{i-1}与p_j得

$$p_1\cdots p_{i-2}\, p_j\, p_i p_{i+1}\cdots p_{j-1}\, p_{i-1}\, p_{j+1}\cdots p_n$$

(4) 反排p_j后面的数得到(q):

$$p_1\cdots p_{i-2}\, p_j\, p_n\cdots p_{j+1} p_{i-1} p_{j-1}\cdots p_{i+1}\, p_i$$

例2.4 设S={1,2,3,4}, 用字典序法求出S的全部排列.

解 1234, 1243, 1324, 1342, 1423, 1432,

2134, 2143, 2314, 2341, 2413, 2431,

3124, 3142, 3214, 3241, 3412, 3421,

4123, 4132, 4213, 4231, 4312, 4321.

思考题: 是否可定义一种序列, 并定义序列之间的一种序关系, 使得可按照序列产生出字典式序排列所有n阶排列.

3. 邻位互换法

- 邻位互换生成算法的思想是很自然的一种想法, 其中蕴涵递归的思想.
- 是由Johnson-Trotter首先提出的.
- 通过把n插入到n-1阶排列的不同位置得到n阶排列:

  n=1: 1

  n=2: 12, 21.

  n=3: 123, 132, 312; 321, 231, 213.

n=4: 1234, 1243, 1423, 4123

4132, 1432, 1342, 1324

3124, 3142, 3412, 4312

4321, 3421, 3241, 3214

2314, 2341, 2431, 4231

4213, 2413, 2143, 2134

- 用这种方法可以产生出任意n阶排列.
- 为了产生n阶排列, 我们必须知道所有n-1阶排列.
- 如果考虑算法, 必须存储所有n-1阶排列, 然后才能产生出所有n阶排列. 这是一个很大的缺点.
- 分析过程, 找到规律, 直接找到通过邻位交换来产生的下一个排列方式.

- 从初始排列1234开始, 在每个数上方加一个箭头“←”, 如$\overset{\leftarrow}{1}\ \overset{\leftarrow}{2}\ \overset{\leftarrow}{3}\ \overset{\leftarrow}{4}$
- 当一个数上方箭头所指的一侧, 相邻的数比这个数小的时候, 称这个数处于活动状态. 在$\overset{\leftarrow}{1}\ \overset{\leftarrow}{2}\ \overset{\leftarrow}{3}\ \overset{\leftarrow}{4}$ 中数2,3,4都处于活动状态.

263154

中6、3、5都是处于活动状态. 显然1永远不是活动的，*n*除了以下两种情形外，它都是处于活动状态的:(1) *n*是第一个数，且其方向指向左侧；(2) *n*是最后一个数，且其方向指向右侧。

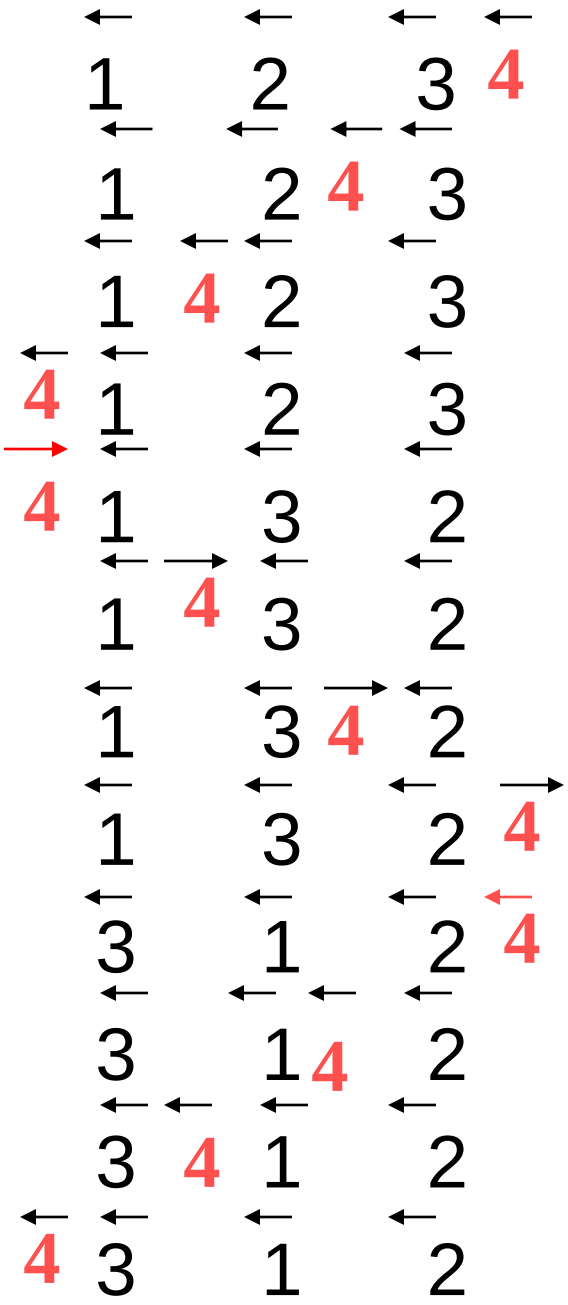
- 利用这个概念可以把上面的生成排列的方法叙述的比较清楚.

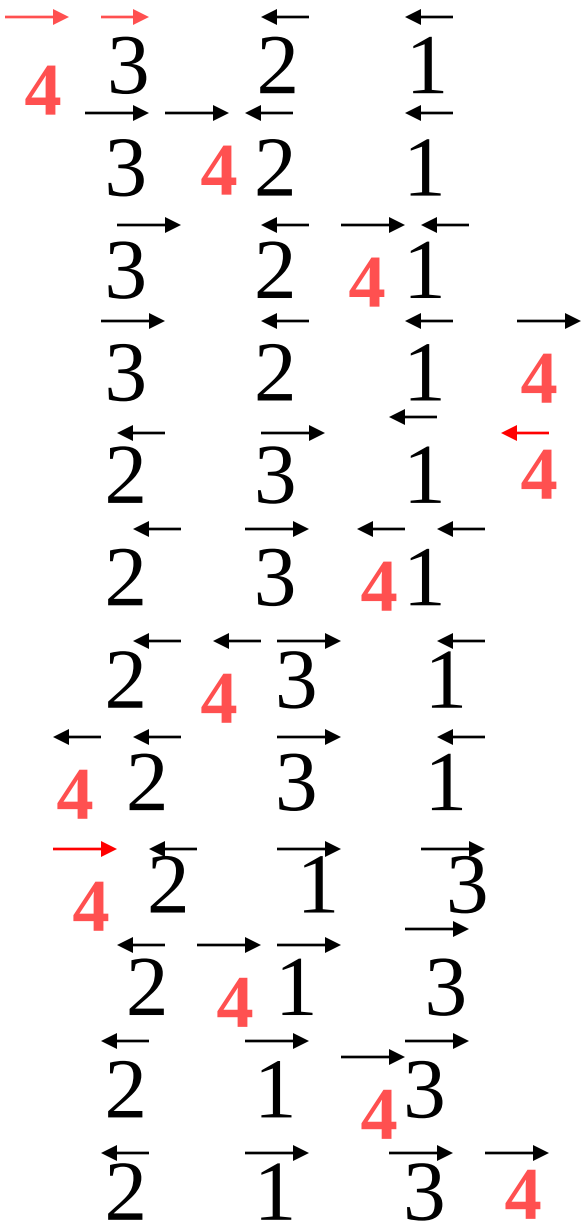
- 设有排列$(p) = p_1 p_2 \cdots p_n$.

Step 1. 若排列$(p) = p_1 p_2 \cdots p_n$中没有处于活动状态的数, 则停止.

Step2. 若排列$(p) = p_1 p_2 \cdots p_n$中有处于活动状态的数, 则设m是处于活动状态数中的最大者. 把m与它箭头方向所指的相邻数互换位置.

Step 3. 改变所有比m大的数上方的箭头; 然后转向**Step 1**.


1 2 3 4
1 2 4 3
1 4 2 3
4 1 2 3
4 1 3 2
1 4 3 2
1 3 4 2
1 3 2 4
3 1 2 4
3 1 4 2
3 4 1 2
4 3 1 2


4 3 2 1
3 4 2 1
3 2 4 1
3 2 1 4
2 3 1 4
2 3 4 1
2 4 3 1
4 2 3 1
4 2 1 3
2 4 1 3
2 1 4 3
2 1 3 4

4. 轮转法

- 轮转法是我国数学家于1996年提出的. 该算法的求解过程如下：
- 给定n个不同元素$N_1,N_2,\ldots,N_{n-1},N_n$, 将$N_1N_2\ldots N_{n-1}N_n$叫做基准排列.

(1) 先逐步生成以N_1打头的所有排列

(a) 先生成以$N_1N_2\ldots N_{n-2}$ 打头的所有排列

$N_1N_2\ldots N_{n-2}\,N_{n-1}N_n$, $N_1N_2\ldots N_{n-2}N_n\,N_{n-1}$

(b) 再生成以$N_1N_2\ldots N_{n-3}$打头的排列.

在**(a)**中生成的两个排列均在其内, 并对它们中的每一排列, 使$N_1N_2\ldots N_{n-3}$不动, 对其后继元素从左向右按顺时针方向轮转两次, 每轮转一次便生成一个新排列, 由此共生成四个新排列, 连同**(a)**中的两个排列共六个排列:

$N_1N_2\ldots N_{n-3}\,N_{n-2}N_{n-1}N_n$

$N_1N_2\ldots N_{n-3}\,N_{n-1}N_nN_{n-2}$

$N_1N_2\ldots N_{n-3}\,N_nN_{n-2}N_{n-1}$

$N_1N_2\ldots\ N_{n-3}\,N_{n-2}N_n\,N_{n-1}$

$N_1N_2\ldots\ N_{n-3}\,N_n\,N_{n-1}N_{n-2}$

$N_1N_2\ldots\ N_{n-3}\,N_{n-1}N_{n-2}\,N_n$

(c) 生成以$N_1N_2\cdots N_{n-4}$打头的所有排列：

在(b)中生成的排列均在其内，并对其中每一排列，保持$N_1N_2\cdots N_{n-4}$不动，使其后继元素从左向右按顺时针方向轮转$n-(n-3)=3$次，每轮转一次便生成一个新排列，共生成18个新排列,连同(b)中的6个排列共24个排列.[省略]

(d) 按照上述方法, 依次分别生成以

$N_1N_2\cdots N_5$, $N_1N_2\cdots N_4$, …, N_1N_2, N_1

打头的所有排列为止.

(2) 生成以N_2打头的所有排列

(a) 先将基准排列$N_1N_2\cdots N_{n-1}N_n$从左向右依顺时针方向轮转一次, 生成排列

$N_2\cdots N_{n-1}N_n\ N_1$

(b)然后按(1)的方法和步骤生成以N_2打头的所有排列.

(3) 生成以N_3打头的所有排列

(a)先将基准排列基准排列$N_1N_2\ldots N_{n-1}N_n$从左向右依顺时针方向轮转两次, 生成以下排列$N_3\ldots N_{n-1}N_n\ N_1\ N_2$

(b)然后按(1)的方法和步骤生成以N_3打头的所有排列.

(4) 依次类推, 按同样方法依次生成以$N_4,N_5,\ldots,N_{n-1},N_n$打头的所有排列.

- 到此整个算法结束.

1234	1243	2341	2314
1342	1432	2413	2143
1423	1324	2134	2431

3412	3421	4123	4132
3124	3214	4231	4321
3241	3142	4312	4213

二. 组合生成算法

- 组合的生成要比排列容易. 我们将给出组合生成的标准算法.
- 先观察从1,2,…,6中任意取3个数的组合.

01	123	02	124	03	125	04	126	05	134
06	135	07	136	08	145	09	146	10	156
11	234	12	235	13	236	14	245	15	246
16	256	17	345	18	346	19	356	20	456

- 上述的组合等于按照字典序排列好了.
- 每个组合$c_1c_2c_3$满足条件$1\le c_1< c_2< c_3\le 6$.
- c_3最大可以到6, c_2最大可以到5, c_1最大可以到4.
- 如果每个数都已经达到最大，那就结束了．如果没有，就找最右面一个还没有达到最大值的数，给这个数依次分别加1,2,…,r-j+1得到下一个组合．重复这个过程就可以得到整个组合.

- 设要按字典序决定集合$S=\{1,2,\ldots,n\}$的全体r组合. 设S的一个r组合为$a_1,a_2,\ldots,a_r$, 且$1\le a_1< a_2< \ldots < a_r \le n$.

① 若$a_1,a_2,\ldots,a_r$等于$n-r+1, n-r+2, \ldots, n$, 则它已经是最后一个组合.

② 若$a_1,a_2,\ldots,a_r$不等于$n-r+1, n-r+2, \ldots, n$, 设i是使$a_j<n-r+j$的最大整数.

③ 则按字典序紧跟$a_1,a_2,\ldots,a_r$的r组合是

$a_1, \ldots, a_{i-1}, a_i+1, a_i+2, \ldots, a_i+(r-i+1)$

- 由这个原理, 从一个组合$a_1, a_2, \ldots, a_r$到下一个组合的算法可以描述如下:

  S1. 求使$a_j<n-r+j$的最大下标i. 即

  $$i=\max\{j|a_j<n-r+j\}$$

  S2. $a_i \leftarrow a_i+1$

  S3. $a_j \leftarrow a_{j-1}+1,\ j=i+1, i+2, \ldots, r.$

例2.6 试生成S={1,2,3,4,5,6,7}的5组合.

解 可以利用上面的算法来生成:

12345, 12346, 12347, 12356, 12357,
12367, 12456, 12457, 12467, 12567,
13456, 13457, 13467, 13567, 14567,
23456, 23457, 23467, 23567, 24567,
34567

其组合数C=(7,5)=21.